Appendix A
Technical Presentation with a Hands-on Example

Please note that in the following discussion we use the term texts to refer to open-ended responses and we use the term topics to refer to codes, but these terms can be used interchangeably.

Latent Dirichlet Allocation. Following Griffiths and Steyvers (2004), in statistical natural language processing, researchers can model the contributions of different codes in a text (i.e., and open-ended response in our case) as probability distributions over words configuring each text. Intuitively, the detection of J codes in a single open-ended response can be expressed as follows:

$$P(w_i) = \sum_{j=1}^{TJ} P(w_i|z_i = j)\, P(z_i = j) \tag{A1}$$

where $P(w_i)$ represents the probability of the ith word appearing in a given open-ended response, z_i is a latent indicator of the code from which the ith word was drawn, and $P(w_i|z_i = j)$ is the probability of the word w_i being identified in the jth code. Finally, $P(z_i = j)$ renders the probability that word i is assigned to code j in the current text, which will vary across different open-ended responses. Griffiths and Steyvers (2004) also indicate that $P(w|z)$ detects which words are important to a code, and $P(z)$ is the code prevalence in a document.

The ideas in Equation (A1) can be extended to a collection of texts, which is referred to as a corpus (as depicted in Figure 3). If one refers to these texts or open-ended responses as D and assumes that they can be classified in J codes given a set of W unique words (or their contribution probabilities), one can represent $P(w|z)$ (where z is a latent code classification to be learned or identified) with a set of J multinomial distributions ϕ over the W words such that the probability of the word w_i being identified in the jth code (or $P(w|z = j)$) can be expressed as $\phi_w^{(j)}$. The code prevalence in each text D also follows a multinomial distribution θ over the J codes such that for a word in text d, $P(z = j) = \theta_j^{(d)}$. With these elements, the latent Dirichlet allocation procedure enables the identification of word relevance/importance per code over the collection of open-ended responses as follows:

$$P(\mathbf{w}|\phi, \alpha) = \int P(\mathbf{w}|\phi, \theta)\, P(\theta|\alpha)\, d\theta \tag{A2}$$

where the goal is to obtain an estimate of ϕ that gives high probability to the words configuring codes that appear in an open-ended response.

Operationally, these distributions are derived from the number of times that a word w has been assigned to code j (or $n_j^{(w)}$) and the number of times a word from text d has been assigned to code j (or $n_j^{(d)}$). Both $n_j^{(w)}$ and $n_j^{(d)}$ are matrices derived from text-mining procedures. In the case of $n_j^{(w)}$, the dimensions are $[j, w]$, where j is the number of rows and accounts for the number of codes, and w is a word index with length obtained from the unduplicated vocabulary contained in the texts. As depicted in Figure B1(a), our vocabulary, based on the HSLS data described in the paper, has a length of 192 terms or words.

Concept Application. To demonstrate how these abstract concepts are applied, let us assume that we have two short texts (same rationale is implemented with tens of thousands of texts):

- "my kids are home, and I cannot focus"

- "I wish I had kids, being home alone is sad and cannot focus"

The analytic process begins by tokenizing each word as follows. We take the first text and replace each word in that text with a numeric identifier starting at 1 and not ending until the entire vocabulary of that text has been covered. Then, the second text is tokenized. Note that the first text has no word repetitions, so its configuring words go from 1 to 8, as shown in Table A1. The second text starts with "I", which was already present in the first text. Accordingly, this word is not repeated as a new column in this table; instead, only words in text 2 that were not part of text 1 are added as new columns in this word index table.

Using this tokenized information, we can then rewrite the original texts using their respective numeric tokens, as depicted in Table A2, without losing information. These tokenized texts can now be mathematically manipulated. Although Table A2 contains the original texts (words), this text information is presented only for clarity purposes; text analyses typically rely on tokenized information. (This tokenization process also works with images, which enables the application of classification techniques to both words and images.)

Initial Random Code Allocation. If we assume that these texts represent two latent codes (more on number of code selection below), we can form a code word matrix (or $n_j^{(w)}$ as defined above) with dimensions $[j, w]$, where j in this case is 2, representing the j number of codes, and w is 14, representing the vocabulary length taken from the number of columns in Table A1. However, without any previous knowledge, the inductive coding process has to begin with code allocations based on random guesses (Griffiths & Steyvers, 2004; Grün & Hornik, 2021). This implies that to populate the matrix $n_j^{(w)}$, the process begins by randomly assigning codes to words in the first iteration of this machine learning classification, as reflected in Table A3.

The code assignment obtained from this initial random code-to-word allocation is used to populate our matrix $n_j^{(w)}$ (Griffiths & Steyvers, 2004; Grün & Hornik, 2021), as depicted in Table A4. Panel A in this table shows the original empty state of each code-to-word allocation. Once the initial iteration is completed, this table is updated as shown in Panel B. The row **colsum** in Panel B is the sum of each column vector and represents the total number of times each token was represented in the entire vocabulary. For example, the token or word "kids" appeared a total to two times, once in each text, and, in this current random assignment example, this word was assigned to both Code 1 and Code 2 once. The ratio of this random assignment to each topic over the **colsum** implies that the word "kids" has a 50% chance of being assigned to Code 1 and 50% chance to being assigned to Code 2, which as of this point in the process is purely based on a random allocation. When applied to each column vector, this table represents a Dirichlet distribution, where the vectors in the sample space of the Dirichlet have the same properties as multinomial probability distributions and can be thought of as a distribution over vectors (or distribution over distributions) (Griffiths & Steyvers, 2004; Grün & Hornik, 2021).

The information contained in Table A3 is also used to build the matrix $n_j^{(d)}$, which has dimensions $[d, j]$. Here d represents the number of texts (2 in this case) and j the number of codes (also 2 in this example). As discussed above, this matrix counts the number of times that a word from text d was assigned to code j. The first iteration of this matrix is shown in Table A5.

The matrix represented in Table A5 shows that of the eight words contained in Text 1, 50% were classified under Code 1 and 50% under Code 2 as a result of this initial random allocation process. In the case of the second text, of the 13 words configuring this response, 38.5% of its words were randomly classified under Code 1. This row vector Dirichlet distribution

of texts to codes is also a distribution over distributions (Griffiths & Steyvers, 2004) based on the number of times the words in a given text were allocated to a given code.

Once more, note that initial probability distributions were based on a random allocation. The machine learning process depicted next will update these allocations of codes to words (Table A3) that will subsequently update Tables A4 and A5. These subsequent updated allocations will be based on prior chained knowledge of their distributions resulting from each immediate previous update of the same codes-to-words assignments. To obtain posterior distributions over the assignment of words to codes, the learning process relies on Bayesian statistics and statistical physics (Griffiths & Steyvers, 2004) via Markov chain Monte Carlo (MCMC) and Gibbs sampling (Grün & Hornik, 2021), as discussed next.

Gibbs Sampling. As just described, the initial allocation of words to codes began with a random process. Accordingly, to obtain the posterior distribution over the assignments of words to codes $P(\mathbf{z}|\mathbf{w})$, we need to train the data iteratively. Griffiths and Steyvers (2004) showed that the latent indicator of the code from which the ith word was drawn (or z_i as defined earlier) depends on all previous $\mathbf{z}_{-i}$values through the counts $n_j^{(w)}$ and $n_j^{(d)}$.

The empirical application of this equation is as follows. Using Gibbs sampling, the full conditional distribution of the next realization of the latent indicator z_i is conditioned on the vector of current code memberships of all words without the ith word w_i. This refers to the **colsum** row in panel B of Table A4. Taking "kids" for example, the condition implies that one should account for all other distribution allocations except for the column vector corresponding to "kids." This is represented as $P(z_i = j|z_{-i}, \mathbf{w})$, and relying on a Markov chain, this equation will be proportional to the multiplication of the probability distributions of words i for codes j (or ϕ_{ij}) and codes j to texts d (or θ_{dj}). Bringing these pieces together, these relations can be represented as:

$$P(z_i = j|z_{-i}, \mathbf{w}) \propto \phi_{ij}\theta_{dj}, \tag{A3}$$

where $P(z_i = j|z_{-i}, \mathbf{w})$ is the posterior probability of identifying code j (in the form $z_i = j$), conditional on previous estimated distributions of all codes and words in the corpus or collection of texts with the exception of z_i. The term $\propto$ indicates that the estimation of these conditional distributions of code assignment is proportional to the product of the probability distributions of words i for codes j (ϕ_{ij}) and codes j to texts d (θ_{dj} as defined above).

The term $\phi_{ij} = \frac{n_{-i,j}^{(w_i)} + \beta}{n_{-i,j}^{(\cdot)} + W\beta}$ where $n_{-i,j}^{(w_i)}$ counts the number of times word i has been assigned to code j with the exception of its current identification ith in text i. To represent these concepts with an actual example, observe that the word "kids" is indexed as the second word in the first text response as shown in Table A2; we will refer to "kids" as w_i henceforth. Note also that in panel B of Table A4, the column vector of "kids," or w_i, has a total value of 2 with one of these realizations belonging to w_i. Accordingly, by removing w_i the count in the column vector kids is reduced by one and thus becomes zero in its intersection with row Code 2 and column "kids" in Table A4 but remains 1 in the realization of w_i in Text 2 in the same table.

The term $n_{-i,j}^{(\cdot)}$ in the denominator is the count of the number of words per each row code vector j. Once more, with the exclusion of w_i from Text 1, the sum of rows for Code 2 will decrease by one given the omission of w_i from the column count in Table A4. W is the length of the unduplicated number of words in the corpus. In our example, this length is 14 (see Table A1). β is a hyperparameter used to control the learning process with a starting value of 0.1 in the form

$\beta \sim$ Dirichlet $\delta = 0.1$ (Griffiths & Steyvers, 2004; Grün & Hornik, 2021). Note that β is a "parameter of the prior distribution for the term distribution of the topics and ..." (Grün & Hornik, 2021, p. 6) and as a starting value it will be continuously updated over the entire classification process. Although the analytic procedures have the option of maintaining this value fixed over the entire analytic process, by selecting not to estimate it as the iterations progress, fixing this hyperparameter is only recommended whit simulated data where the actual parameters are known, rather than learned as part of this estimation process via the MCMC resamplings discussed below or when researchers want to use the resulting or estimated values obtained from a previous model (Grün & Hornik, 2021).

In addition to aiding with the learning process, β allows the numerator to never be zero, even in cases when the removal of w_i renders a value of zero for a given code as in our example. Intuitively, this ratio expresses the probability of w_i to be assigned to j given all other realizations of this token across all texts. Since these procedures are indexed, if the word "kids" had appeared again in Text 1, this count would have still been accounted for in the resulting probabilities.

The last term in Equation (A3) is $\theta_{dj} = \frac{n^{d_i}_{-i,j}+\alpha}{n^{d_i}_{-i}+J\alpha}$, where d_i indicates the text in the corpus to which word w_i belongs. Accordingly, $n^{d_i}_{-i,j}$ counts the number of times a w_i from text i has been assigned to code j, once more excluding its current realization. For consistency, let us once again refer to "kids" in Text 1 to illustrate this process. Note that since this word was assigned to Code 2, its initial realization rendered a value of 4 in Table A5, that indexes the code-to-word allocation. Hence, following the previous rationale, by excluding w_i, the value of Code 2 in Text 1 will decrease from 4 to 3 with no other changes made. The first element of the denominator, $n^{d_i}_{-i}$, counts the total number of words in a text i, with the exception of w_i. Once more, by excluding w_i, the sum of rows for Text 1 in Table A5 will decrease from 8 to 7. J is the number of codes to be estimated and α is another hyperparameter that has been shown to perform well with a standardized value of $\frac{50}{J}$ (Griffiths & Steyvers, 2004; Grün & Hornik, 2021) (more on the optimal code selection number below). Different from the probabilities obtained from ϕ_{ij}, the probabilities obtained from θ_{dj} will add to one per each row in the updated Table A5. Accordingly, at the end of the learning process, one can select the texts or open-ended responses that best represent each topic (exemplary summaries of the contributions of words to codes) as depicted in our conceptual framework and implemented in MDCOR.

As in the case of β, α is also updated throughout the learning process (Grün & Hornik, 2021), and even though it can also be kept fixed, this approach is not implemented by MDCOR for there is no empirical evidence that backs this methodological choice. Moreover, as Grün and Hornik states, as a result of the learning process, only the best model with respect to the log-likelihood $log(p(w|z))$ observed during Gibbs sampling is returned (Grün & Hornik, 2021, p. 13). Having said this, our software application allows users to manually set the starting values of β and α. However, we highly recommend keeping the default values (0.1 and 50 for β and α, respectively) and simply update these values for robustness checks. Moreover, we are not allowing these values to remain fixed, given that MDCOR's use to close open-ended questions is unlikely to include learned hyperparameters from previous models. Having said this, the back-end code (available here https://github.com/democratizing-data-science/MDCOR) may be updated to maintain these values fixed.

Before moving to the next topic, let us note that the value of α of 50, represents the numerator, the denominator will be given by the specific number of topics (or J in the fraction

$\frac{50}{J}$). This value of J will be dynamically updated in the metrics assessment and will be also used in the latent Dirichlet allocation with Gibbs sampling implemented to run the text classification in MDCOR.

In sum, the probabilities ϕ_{ij} and θ_{dj} are learning parameters of code assignments iteratively updated thousands of times. For each word in each text, the algorithm will reassign a new code based on these resulting probabilities just described, ultimately updating Table A3 based on this Gibbs sampling learning process. From this perspective, although z_i latent variables are initiated by randomly assigning codes to words, the MCMC will improve this assignment process over thousands of iterations.

Given that the initial assignment is randomly generated, the first iterations should be discarded. In this respect, Raftery and Lewis (1991) demonstrated that accuracy may be achieved with about 5,000 iterations after discarding 500 initial assignments or burn-in samples. As discussed in the description of our software, the default settings account for 500 burn-in samples and 5,000 Gibbs iterations to compute posterior probabilities (which total 5,500 iterations). Given that we are not simulating data but instead generating new samples based on the information learned from previous samples and their corresponding estimated probability distributions, if no new information is learned during this resampling process, the model will converge or stabilize (Griffiths & Steyvers, 2004); that is, the sample values will have the same distribution as if they were sampled from the true posterior joint distribution. From this view, increasing the number of iterations of Gibbs sampling will generally increase the precision with which the density of the posterior probabilities can be estimated (Raftery & Lewis, 1991). Our application begins with a seed number, so that all computations will render consistent results as a function of burn-in and MCMC/Gibbs resamplings value selection.

Topic Number Identification. As just described, the latent Dirichlet allocation (LDA) begins the sampling process with the random allocation of codes to words across texts. This implies that the number of codes has to be selected before the Dirichlet distributions and the Gibbs sampling procedures can be applied. Hence, a question that has important implications for the end product of the learning process is how to identify the optimal number of codes that may render useful text response classifications?

According to Griffiths and Steyvers (2004), this optimal number of codes can be approximated with "perplexity," a measure of "performance for statistical models of natural language" (Griffiths & Steyvers, 2004, p. 5230). Perplexity measures uncertainty levels of predicting a single word. Recall that the LDA algorithm goes word by word across texts to update the code assignment during the Gibbs resampling process. Accordingly, if this prediction has high levels of uncertainty, code allocation would be similar to relying on noise. However, with lower prediction uncertainty, the results will be robust to random allocation. Hence, lower perplexity values are better.

To address the question and challenge of choosing the appropriate value for J (the number of codes), MDCOR relies on four metrics (Arun et al., 2010; Cao et al., 2009; Deveaud et al., 2014; Griffiths & Steyvers, 2004). These metrics seek to identify the optimal number of codes to improve the quality of the resulting code classification. Although each of these metrics shares the same goal—to find the most adequate number of codes—their approaches differ procedurally (see Arun et al., 2010; Cao et al., 2009; Deveaud et al., 2014; Griffiths & Steyvers, 2004, for details). In two metrics, the optimal number of topics is reached when the overall dissimilarity between codes achieves its maximum value (Deveaud et al., 2014; Griffiths & Steyvers, 2004). If

dissimilarity increases among codes within each LDA call (i.e., a MDCOR analysis using j number of codes), this implies that within this call, the selected number of codes would share little information in common, implying that they would have few shared words, hence allowing a clearer code differentiation across open-ended responses.

The second approach consists of detecting a number of codes wherein the average correlation between each pair of codes, also within a single LDA call, is minimized (Arun et al., 2010; Cao et al., 2009). In this approach, when codes within a LDA call have an average value close to zero, it means that each correlation dyad (i.e., pair of codes) approaches independence, which indicates that there is little to no overlap in the words being shared by this code dyad hence aiding with latent code identification. All of these metrics for identifying the optimal number of codes, then, require executing LDAs with different numbers (J) of codes. Our default approach, as depicted in our software application, consisted of executing these metrics with two to 60 codes. These metrics also require us to specify burn-in and Gibbs re-samplings. Accordingly, for consistency, we also relied on 500 burn-in initial samples and 5,000 iterations. If researchers prefer to adjust these initial values, the LDA execution will account for those modified settings. To execute these metrics, we relied on the R function provided by Nikita and Chaney (2020). As can be seen in Figure B1(b), our software application produces a plot so that researchers may evaluate the result of these metrics visually and select the most optimal number of codes identified.

Text Preparation and Mining

Although MDCOR is currently only available for the English language, the inclusion of other languages is possible and straightforward and can be accomplished with the source code we provide (see source code access on page 68).

Once the collection of open-ended responses (in *.csv format) has been uploaded, the text preparation processes begin with the identification of non-blank responses in the database. After this initial identification, standard text mining procedures are implemented (Feinerer, 2020; Silge & Robinson, 2017). First, all non-English characters such as: “&,” “Ÿ,” “#,” “\r\n” are removed. Additionally, all characters are transformed to lower case to avoid creating duplicate words with the same meaning (e.g., “Learn,” “learn,” and “LEARN”). MDCOR also removes terms classified as “stop words” (e.g., “at,” “by,” “for,” “with,” “on,” “off,” “over,” “under”) that add no meaning by themselves —for clarity, note that our text example in the previous section was not subjected to these transformations.

In addition, MDCOR implements word lemmatization, a text normalization technique typically employed in natural language processing (Silge & Robinson, 2017). This process applies a morphological analysis to words with the goal of reducing variants while preserving the full meaning of a word based on its form (i.e., pronoun, verb, noun, adjective). We selected lemmatization instead of stemming, another normalization technique, because the latter preserves just the root or base of a word, resulting in loss of meaning and usually incomplete words. For example, in the string “educational,” “educate,” “educate,” “education,” “educative,” applying stemming results in “educ,” “educ,” “educ,” “educ,” “educ.” Applying lemmatization to the same original string, renders “educational,” “educate,” “educate,” “education,” “educative.” Lemmatization is more computationally demanding in both the initial transformation and the length/complexity of the resulting vocabulary. That is, in stemming, this example renders one

word (i.e., one column in the word index vocabulary length similar to Table A4) whereas lemmatization yields four words (which translates into four columns, see below).

The frequency of occurrence of words within texts is captured in a document term matrix. Going back to our initial example of two texts, we represent this matrix in Table A6. The documents configure the rows of the matrix, and the terms configure the columns. Note that all words in the first document appeared only once (first eight columns); in the second document the word "I" was used twice, hence its frequency of 2. All words that were not used in a given document appear with a frequency of zero. These frequencies are used to calculate sparsity in a matrix or the prevalence of zeroes, indicating that some words, while appearing in one document, are not part of the vast majority of the remaining documents. Our user interface was programmed to detect texts containing very uncommon words and print their content as part of the output. MDCOR also allows researchers to download these cases for posterior analyses (see example in the software application section).

Finally, following Schütze et al. (2008) we provide researchers with the option of removing the most "extremely" common words across all texts because they potentially add little value in classifying codes (Schütze et al., 2008). This potential issue may be particularly true when these extremely common words are synonyms of the open-ended question or prompt, for example. That is, if researchers are interested in *college readiness* issues and the question consists of asking participants to define or explain this concept, the resulting open-ended responses may all include common words such as "college" and "ready." If researchers believe that keeping these words in the corpus would add little value for classification purposes or learn the latent meanings that participants' responses conveyed, then they should remove them. Our user interface shows the top 10 most common words, along with their frequencies across all documents with the option of removing x number of them. This removal option completes the text preparation and mining procedures. Note however, that the removal of words should consider their context in the open-ended responses. In our example we did not remove any word and the resulting seven codes performed remarkably well. If researchers decided to remove words, they should have a clear rationale and may test if this removal impacted the final classification outcomes.

Software and Source Code Access

Mac users can download MDCOR here https://cutt.ly/xv6nnuN, PC users here https://cutt.ly/Lv6nTLG. To execute the application, (a) extract the zip folder, (b) open the folder starting with "electron quick start", and (c) double click on the file called "electron-quick-start." The first time MDCOR is executed, researchers may be required to select menu "View" and "Force Reload" to generate all paths to this newly extracted folder. If after forcing reload, MDCOR does not display as shown in Figure 4, close the window, execute MDCOR again, and, if needed, force reload once more. The interface will now load. This process is only needed the first time MDCOR is launched in a computer. The software applications have been comprehensively tested for viruses and are housed in a secure server location. Accordingly, all executable files are virus-free but may require administrative permission to be deployed. For the execution of the metrics assessment (step five in Figure 4), Windows requires access to parallelization to speed up the process. This process will also request administrative permission the first time MDCOR is executed in a Windows machine (this process is not required in Mac). Finally, each of these executable files for the Mac and Windows operative systems contain all the source code required

to modify and/or expand the application capabilities in the file called “app.R”.

Table A1
Word index with vocabulary length

my	kids	are	home	and	I	cannot	focus	wish	had	being	alone	is	sad
1	2	3	4	5	6	7	8	9	10	11	12	13	14

Table A2
Rewriting of original texts with their corresponding tokenized words

Panel A. First text												
my	kids	are	home	and	I	cannot	focus					
1	2	3	4	5	6	7	8					
Panel B. Second text												
I	wish	I	had	kids	being	home	alone	is	sad	and	cannot	focus
6	9	6	10	2	11	4	12	13	14	5	7	8

Table A3
Random assignment of j codes to words in each text

Panel A. First text													
	my	kids	are	home	and	I	cannot	focus					
RTA*	2	2	1	1	2	2	1	1					
Panel B. Second text													
	I	wish	I	had	kids	being	home	alone	is	sad	and	cannot	focus
RTA*	2	2	1	1	1	2	2	2	1	2	2	1	2

*Random topic allocation or assignment

Table A4
Word to code assignment, initial and first iteration of matrix $n_j^{(w)}$

Panel A. Original stage														
jth code	my	kids	are	home	and	I	cannot	focus	wish	had	being	alone	is	sad
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Panel B. Initial iteration, random assignment														
jth code	my	kids	are	home	and	I	cannot	focus	wish	had	being	alone	is	sad
1	0	1	1	1	0	1	2	1	0	1	0	0	1	0
2	1	1	0	1	2	2	0	1	1	0	1	1	0	1
colsum	**1**	**2**	**1**	**2**	**2**	**3**	**2**	**2**	**1**	**1**	**1**	**1**	**1**	**1**

Table A5
First iteration of matrix $n_j^{(d)}$ *fed from a random code to word allocation depicted in Table A3*

	Code 1	**Code 2**
Text 1	4 (50%)	4 (50%)
Text 2	5 (38.5%)	8 (61.5%)

Table A6
Document Term Matrix

vocabulary	my	kids	are	home	and	I	cannot	focus	wish	had	being	alone	is	sad
1	1	1	1	1	1	1	1	1	0	0	0	0	0	0
2	0	1	0	1	1	2	1	1	1	1	1	1	1	1

Appendix B
Executing MDCOR With Restricted Data

This section relies on 25,206 open-ended responses taken from the protected version of the NCES' High School Longitudinal Study. This database is the most recent nationally representative sample of high school students transitioning to college and to the job market. We analyzed the protected version of HSLS because the public version does not contain open-ended responses. Given that MDCOR runs locally and does not require internet access, it complies with NCES security standards for protected data. Another reason to choose HSLS data is to test the performance of MDCOR when analyzing short texts. Specifically, the open-ended question in the HSLS survey asked legal guardians or parents to describe the primary activities their main employment requires. As Albalawi et al. (2020) and Li et al. (2016) stated, the analyses of short texts is more demanding and challenging than the analysis of long texts due to sparsity or uniqueness issues (see below). Accordingly, if MDCOR's procedures render meaningful codes with short texts, these results would validate its usefulness in analyzing challenging open-coded responses.

Text Mining

Figure B1(a) shows the outputs corresponding to Steps 1 through 3. After the data are uploaded, the *ID* and *Text* boxes will be populated with the column names of the data selected. The feedback after executing Step 1 indicates the number of rows and columns (i.e., the dimensions) of the uploaded database (25,206 rows and 28 columns in HSLS). This information does not yet account for any blank or coded nonresponses. As discussed above, NCES provides coded responses for *blank/nonresponses*. Accordingly, the number of responses will match the first feedback provided after Step 2 is executed. This feedback will also present the number of responses dropped due to *sparsity or uniqueness* levels; researchers can download dropped cases and merge them with the original data for further analyses. In this output, the maximum sparsity of the matrix to be able to classify text data is 99%, which indicates that all remaining 13,140 open-ended responses to be classified will have terms appearing in at least 1% of all responses. Note that single-word responses are likely to be dropped if they do not provide enough information to classify the response into a given code. For example, the response "CLERK" was dropped due to lack of context to help its classification. This response should be carefully assessed and, if pertinent, added to the resulting machine-learned codes (discussed below).

This output also shows that the number of terms or vocabulary (a product of the lemmatization procedure discussed in the appendix) is 192. The 10 most frequent words are shown in this output as well. The optional removal of the most frequent words is shown in Step 3 (trimming words). In our example, we did not remove any words but still executed this step to showcase MDCOR's behavior. Because we did not remove any words, the output mirrors the document term matrix and common words information shown in Step 2.

Metrics Assessment

Figure B1(b) shows the results of Steps 4 and 5 in Figure 4. Specifically, the resulting metrics plot shows the four assessments of the optimal number of codes associated with all 13,140 open-ended responses. Because Step 3 in Figure 4 is optional, the metrics plot may be based on either the complete or the trimmed document term matrix.[9]

In our example, we executed the metrics with the recommended 500 burn-in samples and 5,000 MCMC/Gibbs resamplings (Raftery & Lewis, 1991). Ideally, all four metrics would converge into a meaningful number of optimal codes; however, based on text complexity, the convergence of at least two metrics may suffice to make an informed decision regarding the optimal number of codes that capture meaningful information (see appendix, and Nikita and Chaney (2020)). The resulting plot indicated that, based on metrics by Deveaud et al. (2014) and Cao et al. (2009), the optimal number of topics is between seven and 11, as can be seen by the maximization and minimization indicators, respectively (we added the red oval for clarity). In this example we selected the lowest number, but as discussed below, we strongly recommend conducting sensitivity analyses by also selecting either the midpoint between the two metrics or the largest number to see how these selections impact the meaning of the resulting codes (see Nikita & Chaney, 2020).

Code Modeling

Based on the metrics assessment, the number of codes in Step 6 was adjusted to 7, and Figure B2 shows the result of executing MDCOR (Step 7). Figure B2(a) shows a table with the two most representative codes given their relative contribution (recall that up to 20 of the most representative responses per code can be displayed and downloaded), the code groups with the highest values, and the original quote or text input. In our example, the most representative responses per code are sorted by relative group fit and by code. Relative group fit is a measure of code coherence, or how frequently the words configuring this code appear together in that latent code. In this sense, Code "V3" has the clearest and most cohesive word frequency across all seven codes. Researchers can easily sort column values directly in the user interface by clicking the arrows next to each column name. They can also search specific words in the search box.

The column "relative_text_contribution" ranges from 0 to 1, with 1 indicating that the text in the column "quote" is the most representative of a given code. Accordingly, this output also shows that the response of participant ID "10925" is the most representative content of Code "V3" across all 13,140 responses in our database. In other words, the content of this text had the highest probability of representing Code "V3" across all other texts (see appendix).

As can be assessed in our output, the classification performed remarkably well. Code "V3" represents health care duties, "V7" computer and software design actions, "V6" office administrative assistant tasks, "V2" teaching/school activities, "V5" managerial tasks, "V4" cleaning activities, and "V1" payroll, real state, and human resources duties. Going back to the word "CLERK" discussed above, without further context, it is impossible to classify this response without making unwarranted assumptions, which explains its removal from these classification procedures.

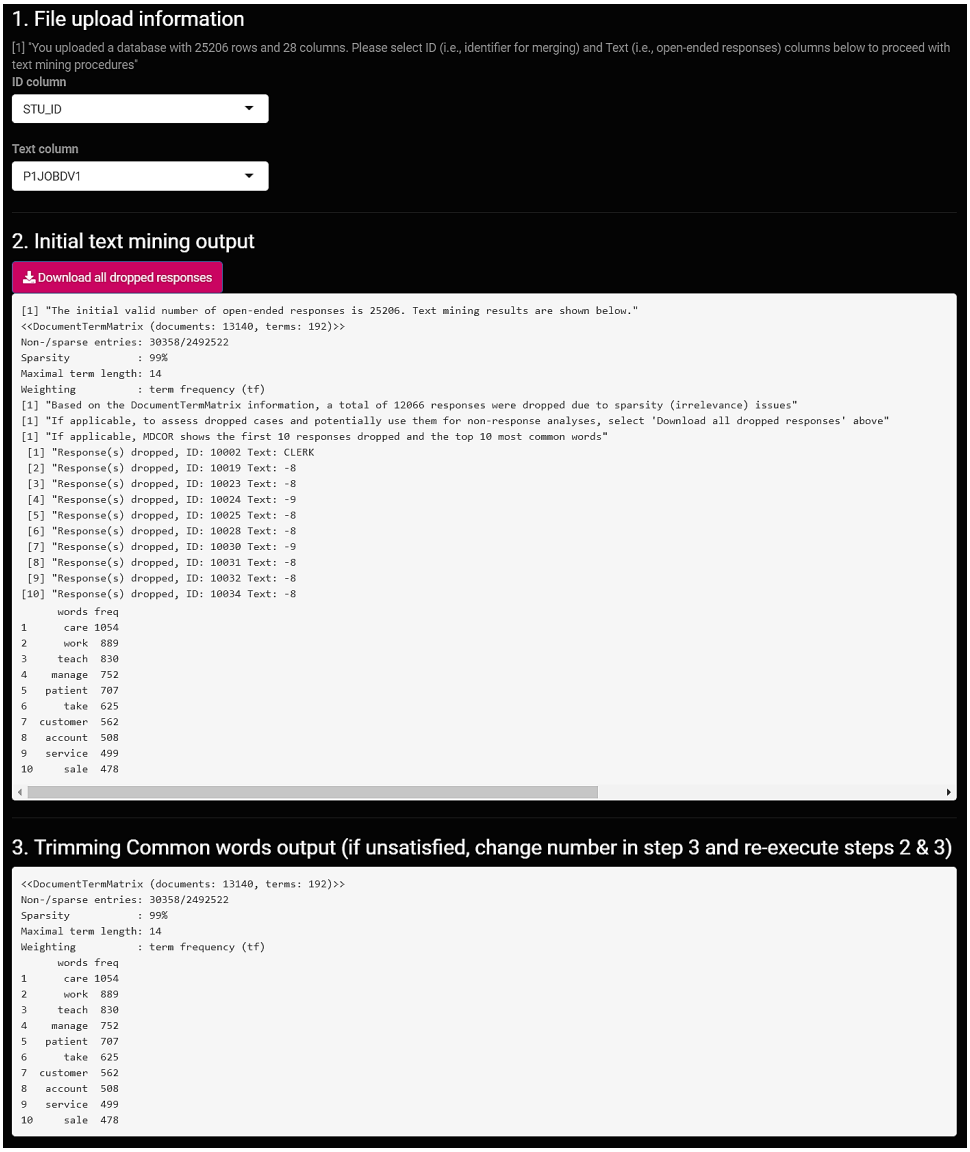
Note that we requested the top 2 most representative cases per code, which should have indicated 14 responses. However, since for Code "V1" there are three responses with the same probabilities tied in second place, this code appeared four times. Nonetheless, this extra information was useful in strengthening our understandings of its latent meaning.

Summary Statistics per Latent Code

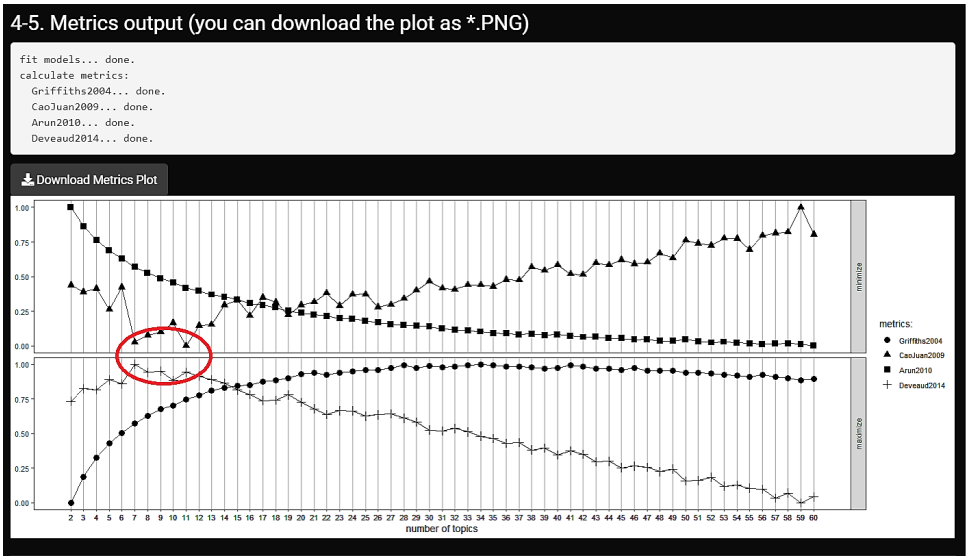
As part of the integrative goal, MDCOR also provides a visual and interactive representation of the most frequent words configuring each code. This is offered as a tool to

broadly understand the meaning of these codes. This word frequency distribution is shown in Figure B2(b). When MDCOR is used with nonrestricted databases, the resulting summary can be hosted online. Because HSLS is federally protected, the interactive version of Figure B2(b) is available only locally.

As described in the main content of the paper (see Figure 3), the distance among codes depicted in Figure B2(b) indicates the level or similarity of their latent meanings based on their probabilities of belonging to a given code. For example, Codes 7 and 2 in Figure B2(b), are closer in meaning than Codes 7 and 1. Code 3 encompasses shared meanings, hence its central place in this distance map. The code bubbles are larger for whatever word is selected in the right side of the figure, a feature that makes this descriptive analysis more interactive.



(a) Text upload, decomposition, and mining



(b) Metrics Plot

Figure B1
MDCOR Outputs, Steps 1 to 5

6-7. Code modeling and most representative texts per code

After MDCOR completion you can download the resulting MLTC datasets

Download Full MDCOR data (all text chunks) | Download Most Representative cases (shown below)

Select the number of most representative texts per code

1 2 20

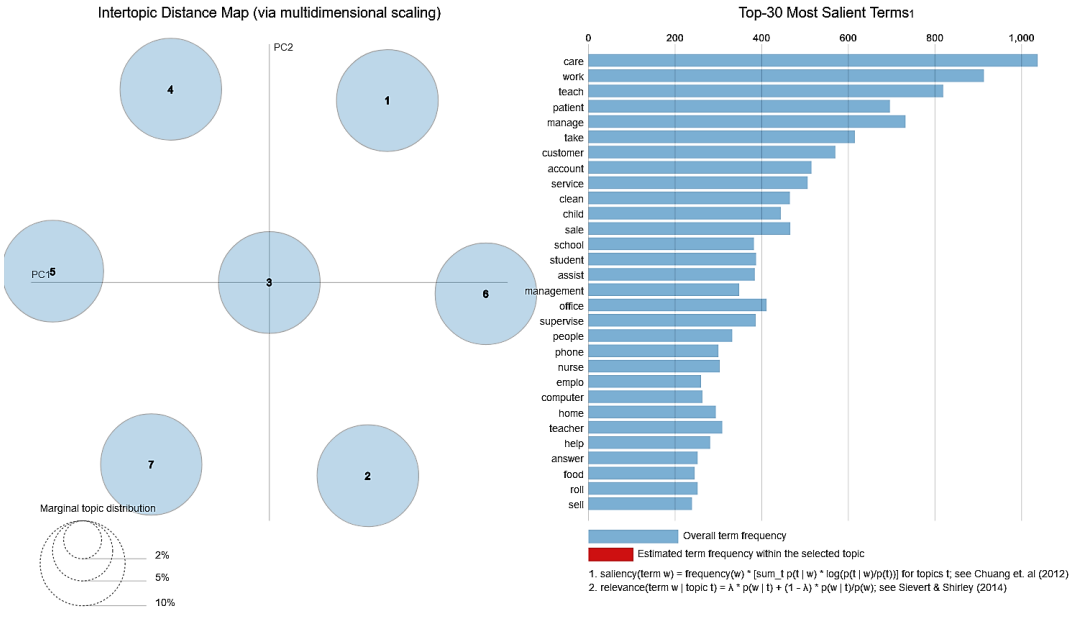
1 3 5 7 9 11 13 15 17 19 20

Show entries | Search:

	code	text	relative_text_contribution	relative_group_fit	quote
1	V3	10925	1	1	TAKE CARE OF PATITNENTS GIVE MEDICATION AND TAKE CARE OF NEEDS IN HOSPITAL
5	V3	18708	0.95	1	CARE FOR PATIENT, TAKE CARE OF INJURIES, TAKE CARE OF FALLS, MEDICATIONS
7	V7	21237	1	0.999	COMPUTER APPLICATIONS SUPPORT; PROJECT MANAGEMENT; REPORTS DEVELOPMENT
8	V7	21340	1	0.999	DESIGN, DEVELOP, TEST AND MAINTAIN COMPUTER SOFTWARE APPLICATIONS
9	V6	21814	1	0.995	ANSWER PHONES, SCHEDULE APPOINTMENTS, FILE PAPER WORK, PREPARE CHARTS,
11	V6	26663	0.968	0.995	TYPING, ANSWERING PHONES TO ASSIST WITH CUSTOMERS TAX PROBLEMS, FILING, DATA ENTRY WORK
10	V2	24892	0.97	0.992	TEACH 6TH GRADE STUDENTS IN READING, MATH, SCIENCE, AND SOCIAL SCIENCE
15	V2	33137	1	0.992	TEAM TEACH WITH A CERTIFIED SPECIAL EDUCATION TEACHER IN A HIGH SCHOOL SPECIAL EDUCATION CLASSROOM
3	V5	15278	1	0.979	SUPERVISE OFFICE STAFF, OVERSEE OFFICE OPERATIONS, MANAGE BUSINESS AFFAIRS FOR PHYSICIAN, PROVIDER CONTRACTING HIRING/FIRING, ETC.
13	V5	30449	0.996	0.979	MANAGE EMPLOYEE AND SHOWROOM'S OVERALL APPEARANCE. BUSINESS MANAGER FOR ALL ASPECTS OF THE DAILY OPERATIONS.
2	V4	11664	0.949	0.969	MAKEING SURE WE HAVE HOME TO CLEAN EVERYDAY & MAKE SURE GIRLS HAVE ALL THE SUPPLIES THEY NEED TO CLEAN THE HOMES. AND IF CLIENTS HAVE ANY PROBLEMS THEY CONTACT ME.
4	V4	18368	1	0.969	CLEANING, PREPARING FOOD, AND OPERATING CASH REGISTER
6	V1	19189	1	0.963	ALL RECORD KEEPING, PAYROLL, ACCOUNTS RECEIVABLE, ACCOUNTS PAYABLE, HUMAN RESOURCES, P & L REPORTS, MEETING/TRAVEL/ENTERTAINMENT PLANNING
12	V1	28571	0.948	0.963	APPRAISE REAL ESTATE FOR ESTATES & FOR MORTGAGES, & SELL REAL ESTATE
14	V1	31730	0.948	0.963	PAYROLL ACCOUNTS RECEIVABLE ACCOUNTS PAYABLE BANK RECONCILIATIONS ETC...
16	V1	34613	0.948	0.963	PAYROLL, ACCOUNTS RECIEVABLE, ACCOUNT PAYABLE, CUSTOMER SERVICE

Showing 1 to 16 of 16 entries | Previous 1 Next

(a) Example of code identification and original text inputs



(b) Summaries of each latent code

Figure B2

MDCOR Outputs, Steps 6 and 7 and summary of codes.